National University of Computer and Emerging Sciences



Lab Manuals

for

Data Structures Lab (CL-2001)

Course Instructor	Mr. Waqas Rana
Lab Instructor	Ms. Fatima Ali
Lab TA	Mr. Abdur Rehman
Section	BSE-3B
Semester	Fall 2026

Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab Manual 06

Objectives:

- Understand recursion: base case, recursive case and how the call stack works.
- Solve classic problems recursively.

Recursion

Recursion is a technique in which a function solves a problem by calling **itself** on a smaller version of the same problem. Each call works on a smaller input until the input becomes so small that the answer is known directly.

Think of standing between two mirrors: you see a smaller copy of yourself inside every reflection, and the copies keep getting smaller until they disappear. A recursive function behaves the same way, it keeps calling a smaller copy of itself until it reaches a stopping point.

Parts of a Recursive Function

- **Base Case:** the simplest input whose answer is known without any further call. It stops the recursion. Without a base case the function calls itself forever and the program crashes with a **stack overflow**.
- **Recursive Case:** the function calls itself with a smaller / simpler input, and uses the returned answer to build the answer for the current input.

Example: Factorial

$$n! = n \times (n - 1)! \quad \text{and} \quad 0! = 1! = 1$$

```
int factorial(int n) {
    if (n <= 1)                     // base case
        return 1;
    return n * factorial(n - 1);    // recursive case
}
```

How Recursion Works: The Call Stack

Every function call is pushed onto the **call stack** together with its own copy of local variables. The calls keep going **down** until the base case is reached (winding). Then the calls return one by one and their results travel back **up** the stack (unwinding).

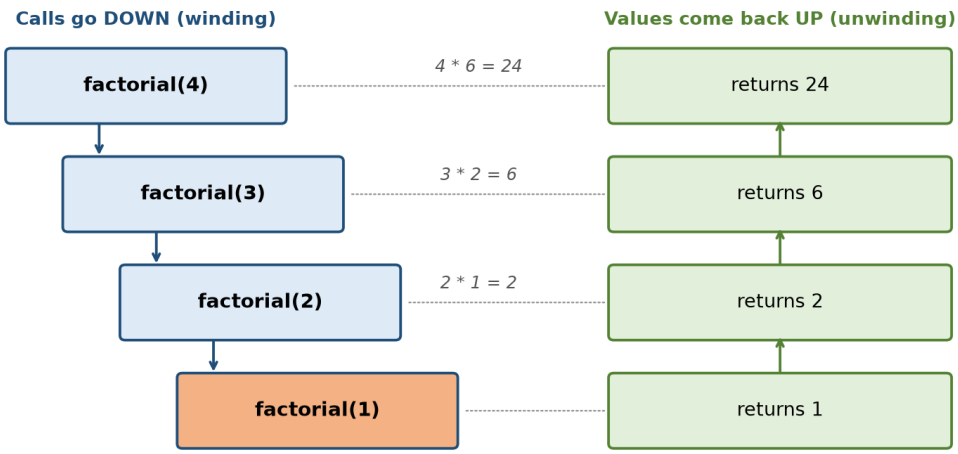


Figure 1: Call stack for factorial(4). The base case (orange) stops the recursion.

Types of Recursion

- **Direct recursion:** a function calls itself, e.g. factorial().
- **Indirect recursion:** function A calls B and B calls A, e.g. isEven(n) calls isOdd(n - 1).
- **Tail recursion:** the recursive call is the last statement of the function (nothing is left to do after it returns).
- **Head recursion:** the recursive call is the first statement; the work is done while returning.
- **Tree recursion:** the function calls itself more than once, e.g. fib(n) = fib(n - 1) + fib(n - 2).

Recursion vs Iteration

Aspect	Recursion	Iteration (Loop)
Termination	Base case	Loop condition becomes false
Memory	Uses call stack (one frame per call)	Constant extra memory
Code	Short and close to the definition	Can be longer for tree-like problems
Risk	Stack overflow if no / wrong base case	Infinite loop if condition never fails
Best for	Trees, divide & conquer, backtracking	Simple repetition, counting

Lab Tasks

Task 1: Recursion Warm-up (5)

Write the following functions using **recursion only** (no loops are allowed). Call each function from main() and print the result.

a) Sum of Digits
Input: `n = 1234`

Output: `10`
Explanation: 1 + 2 + 3 + 4 = 10

b) Power of a Number
Input: `x = 2, n = 10`

Output: `1024`

c) Palindrome String
Write bool isPalindrome(string s, int left, int right).

Input: `s = "racecar"`
Output: `true`

Input: `s = "fast"`
Output: `false`

d) Reverse String

Reverse a character array **in place** by swapping the two ends and recursing on the inner part.

Input: `s = ["h", "e", "l", "l", "o"]`
Output: `["o", "l", "l", "e", "h"]`

Task 2: Tower of Hanoi (5)

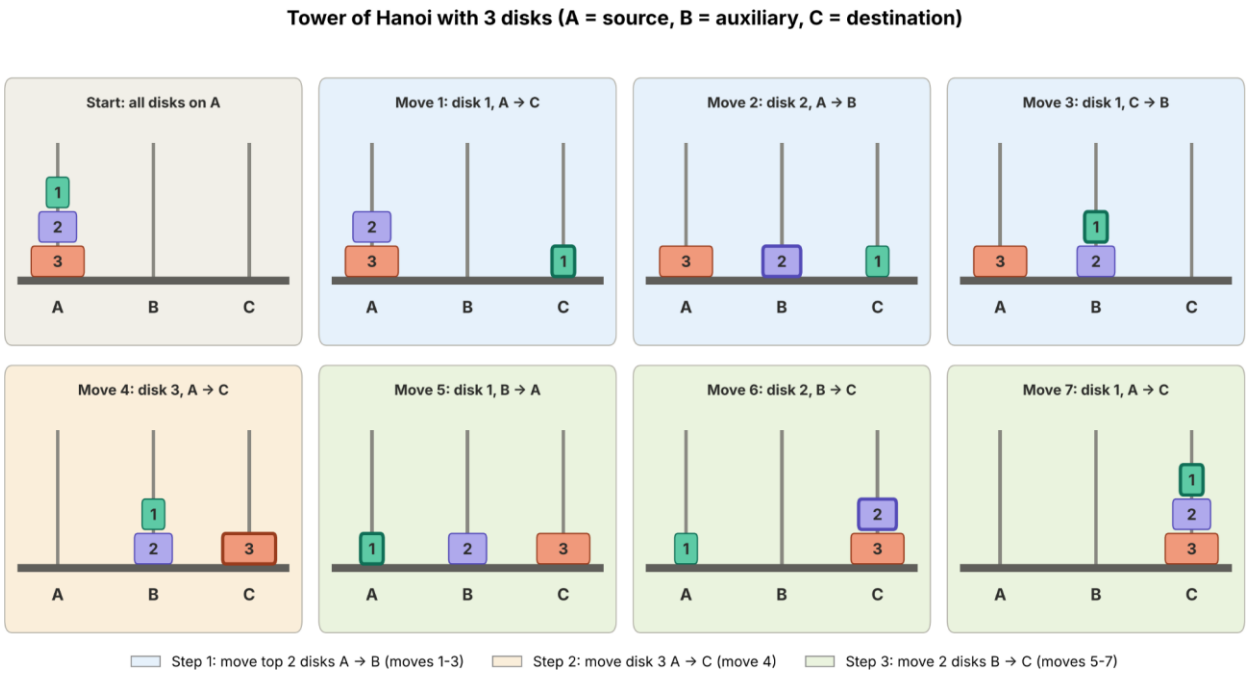
There are three rods named **A** (source), **B** (auxiliary) and **C** (destination). Rod A holds n disks of different sizes, the largest at the bottom. Move all disks to rod C following these rules:

- Only one disk can be moved at a time.
- Only the top disk of a rod can be moved.
- A larger disk can never be placed on top of a smaller disk.

Write a recursive function void hanoi(int n, char from, char to, char aux) that prints every move, and finally print the total number of moves (which must equal 2^n - 1).

Sample Run (n = 3)

```
Enter number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves: 7
```



Task 3: Generate All Subsets (5)

Given an array of unique integers, print all possible subsets (the power set) using recursion. The solution must not contain duplicate subsets and may be printed in any order.

Input: `nums = [1, 2, 3]`
Output: `[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]`
Explanation: An array of n elements has 2^n subsets, so 2^3 = 8 subsets are printed.

Submission Guidelines

Submit only the following file, strictly following the naming convention (replace the roll number with your own):

- l261234.docx (screenshots of the output of every task)
- Copied / plagiarised code will be awarded zero marks.